

P0546r2 : Span - foundation for the future

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0546r2
Date: 2018-02-11
Reply-to: hcedwar@sandia.gov
Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Daniel Sunderland
Contact: dsunder@sandia.gov
Audience: Library Evolution Working Group (LEWG)
URL: https://github.com/kokkos/array_ref/blob/master/proposals/P0546.rst

Revision History	
P0546r1	Update subspan bounds requires clause. Add precedence background information.
P0546r2	Update AccessProperties to Properties. Reference P0900 on properties ontology. Request LEWG straw polls at 2018-03-Jacksonville.

References	
P0009	Multidimensional array reference specification
P0122	span: bounds-safe views for sequences of objects
P0454	Wording for a Minimal <code>mdspan</code>
P0332	Relaxed array declaration
P0367	Accessors
P0900	Property Ontology

Requested Action by LEWG at 2018-03-Jacksonville

- Recall [LEWG 2017-11-Albuquerque](#) straw polls

We should specify the dynamic extent as the element type of the first template parameter rather than the (current) second template parameter

SF 5 , F 3 , N 2 , A 2 , SA 0

We support the addition of access properties variadic template parameters

SF 0 , F 10 , N 1 , A 5 , SA 0

- LEWG 2018-03-Jacksonville requested straw polls:

- Revise P0122 `span` to declare extent as part of the element type of the first template parameter; before progressing `span` through LWG?
 - This modifies `span` to have a single template argument.
- For the single template argument `span` add the properties parameter pack to enable properties such as `atomic_access` (P0860) and `restrict_access` (P0856); before progressing `span` through LWG?
- Introduce into wording the vocabulary of *domain index space* for the set of admissible indices and *codomain element space* for the set of referenced objects.

Motivation

The `span` capability proposed in **P0122, `span: bounds-safe views for sequences of objects`** is intended to provide high performance access to a sequence of elements. The current **P0122** proposal defines two access mechanisms: mapping an integral offset to an element and an iterator over the sequence of elements. The proposed `span` abstraction is the foundation for an extensible set of high performance computing access abstractions, with the proper preparation. This paper proposes

improvements to the **P0122** proposed `span` that will establish a solid, future-proof, and extensible foundation to incorporate additional high performance access needs such as those identified in **P0009** and **P0367**.

Summary of Proposed Changes

```
template < class ArrayType , typename ... Properties >
class span ;
```

1. Instead of `ElementType` provide an `ArrayType` and delete the *magic value* `dynamic_extent` .
 - `ArrayType` is either
 - `ElementType[N]` for a `span` with explicit static length,
 - `ElementType[]` for a `span` with an explicit dynamic length, or
 - `ElementType` for a `span` with an implied dynamic length.

Thus the *current* proposal requires `rank_v<ArrayType> <= 1`.

 - The need for a `dynamic_extent` *magic value* is entirely eliminated.
2. Add properties parameter pack.
 - `Properties...` is a well-defined extension point **P0900**. This extension point, while not exercised in this proposal, enables code which is templated on `span` to be *future proofed* with respect to extensions by including the `Properties` parameter pack. For example this proposal requires `sizeof...(Properties) == 0`; however, the proposed `span` comparison operator overloads retain the `Properties` parameter pack so that they are future proof.
3. Introduce vocabulary of *domain index space* and *codomain element space* .

Precedence

There is solid precedence for declaring static and dynamic array template arguments as `ElementType[N]` and `ElementType[]`

- `std::shared_ptr<T[]>` and `std::unique_ptr<T[]>` denote a dynamic extent array through the incomplete type `T[]`
- P0674 denotes `make_shared<T[] [N1] [N2]>` to allocate a `shared_ptr` to a C style multidimensional array.

Foundation for the Future

The proposed changes prepare `span` to be an extensible foundation for future higher performance intentions. Examples of possible extensions are as follows.

- `ArrayType` could allow (`rank_v<ArrayType> > 1`)
- `Properties...` could include a layout mapping specification when (`rank_v<ArrayType> > 1`) .
- `Properties...` could include a contract specification for *restrict* that guarantees that a `span` type is free from certain forms of aliasing.
- `Properties...` could include memory performance requests such as non-caching loads/stores, streaming access, write only, or random access. **P0367 Accessors** includes numerous potential memory access properties of interest to high performance / low latency computing and heterogeneous architectures.

Proposed Wording Change

Changes to Header `<span>` synopsis

```
namespace std {

// eliminate: constexpr ptrdiff_t dynamic_extent = -1;

template< class ArrayType , typename ... Properties >
class span;

template< class lhsArrayType , typename ... lhsProperties
          , class rhsArrayType , typename ... rhsProperties >
constexpr bool operator==( const span<lhsArrayType,lhsProperties...> &
                           , const span<rhsArrayType,rhsProperties...> & );

template< class lhsArrayType , typename ... lhsProperties
```

```

        , class rhsArrayType , typename ... rhsProperties >
constexpr bool operator!=( const span<lhsArrayType,lhsProperties...> &
                           , const span<rhsArrayType,rhsProperties...> & );

template< class lhsArrayType , typename ... lhsProperties
        , class rhsArrayType , typename ... rhsProperties >
constexpr bool operator< ( const span<lhsArrayType,lhsProperties...> &
                           , const span<rhsArrayType,rhsProperties...> & );

template< class lhsArrayType , typename ... lhsProperties
        , class rhsArrayType , typename ... rhsProperties >
constexpr bool operator<=( const span<lhsArrayType,lhsProperties...> &
                           , const span<rhsArrayType,rhsProperties...> & );

template< class lhsArrayType , typename ... lhsProperties
        , class rhsArrayType , typename ... rhsProperties >
constexpr bool operator> ( const span<lhsArrayType,lhsProperties...> &
                           , const span<rhsArrayType,rhsProperties...> & );

template< class lhsArrayType , typename ... lhsProperties
        , class rhsArrayType , typename ... rhsProperties >
constexpr bool operator>=( const span<lhsArrayType,lhsProperties...> &
                           , const span<rhsArrayType,rhsProperties...> & );

// Note: Prefer output element type of std::byte versus char.
// Note: Static length specification may change for ( rank_v<ArrayType> > 1 )
template< class ArrayType , typename ... Properties >
    conditional_t< ( rank_v<ArrayType> <= 1 ) && ( extent_v<ArrayType> > 0 )
        , span<char[sizeof(remove_all_extents_t<ArrayType>)*extent_v<ArrayType>]>
        , span<char[]>
    >
as_writable_bytes( const span<ArrayType,Properties...> & ) noexcept ;
}

```

Changes to Class template span [views.span]

1 span provides mechanisms to access members within a contiguous sequence (array) of objects. Note that a span does not own the storage of that sequence.

2 Requires: `remove_all_extents_t<ArrayType>` is a complete object type that is not an abstract class type.

`rank_v<ArrayType> <= 1`, which may be relaxed in a future extension of span.

`sizeof...(Properties) == 0`, which may be relaxed in a future extension of span.

3 span has a *domain index space* and a *codomain element space*. The domain index space is the integral values $[0..extent())$. The codomain element space is the contiguous sequence of objects. When `rank_v<ArrayType> > 1` is permitted in a future extension of span the domain index space will be the Cartesian product of integral extents.

4 The iterator type for span is a random access iterator and a contiguous iterator. The `reverse_iterator` type is a random access iterator. Iteration is over the codomain.

```

namespace std {

// Unless noted here, previously proposed span members remain unchanged

template< class ArrayType , typename ... Properties >
class span {
public:
    using element_type = remove_all_extents_t<ArrayType> ;

    // replace: extent

    constexpr static index_type rank() noexcept ;
    constexpr static index_type static_extent() noexcept ;
    constexpr index_type extent() const noexcept ;

    // replace: type-morphing constructors

    template< class OtherArrayType , typename ... OtherProperties >
        constexpr span( const span<OtherArrayType,OtherProperties...> & );

    template< class OtherArrayType , typename ... OtherProperties >
        constexpr span( span<OtherArrayType,OtherProperties...> && );

    // replace: subspan functions

```

```

template< ptrdiff_t Count >
constexpr span< element_type[ Count ], Properties... > first() const ;

template< ptrdiff_t Count >
constexpr span< element_type[ Count ], Properties... > last() const ;

constexpr span< element_type[], Properties... > first( index_type count ) const ;

constexpr span< element_type[], Properties... > last( index_type count ) const ;

constexpr span< element_type[], Properties... > subspan( index_type offset , index_type count = -1 ) const ;
};

```

Changes to span constructors, copy, assignment, and destructor [span.cons]

```
constexpr span() noexcept ;
```

Requires: static_extent() == 0

```
constexpr span( pointer ptr , index_type count ) noexcept ;
```

Requires: static_extent() == 0 OR static_extent() == count.

If ptr is null then count == 0.

If ptr is not null then it shall point to the beginning of a valid sequence of objects of at least count length.

```
constexpr span( pointer firstElem , pointer lastElem ) noexcept ;
```

Requires: distance(firstElem,lastElem) >= 0.

static_extent() == 0 OR static_extent() == distance(firstElem,lastElem).

```

template< size_t N >
constexpr span( element_type (&arr)[N] ) noexcept ;
template< size_t N >
constexpr span( array<element_type,N> & arr ) noexcept ;
template< size_t N >
constexpr span( array<remove_const_t<element_type>,N> & arr ) noexcept ;

```

Requires: static_extent() == 0 OR static_extent() == N.

```

template< class OtherArrayType , typename ... OtherProperties >
constexpr span( const span<OtherArrayType,OtherProperties...> & other );

```

```

template< class OtherArrayType , typename ... OtherProperties >
constexpr span( span<OtherArrayType,OtherProperties...> && other );

```

Requires: static_extent() == 0 OR static_extent() == other.size().

is_same_v< element_type , remove_extent_t< OtherArrayType > >

Effects: Constructs span on the same sequence of objects referenced by other.

Changes to span subviews [span.sub]

```

template< size_t Count >
constexpr span< element_type[Count] , Properties... > first() const ;

```

```

template< size_t Count >
constexpr span< element_type[Count] , Properties... > last() const ;

```

Requires: 0 < Count && Count <= size().

rank() == 1.

```
constexpr span< element_type[] , Properties... > first( index_type count ) const ;
```

```
constexpr span< element_type[] , Properties... > last( index_type count ) const ;
```

Requires: Count <= size().rank() == 1.

```
constexpr span< element_type[] , Properties... > subspan( index_type offset , index_type count ) const ;
```

Requires: 0 <= offset && offset <= size().

count == -1 || offset + count <= size().

rank() == 1.

Returns: `span( data() + offset , ( count == -1 ? size() - offset : count ) )`

Changes to span observers [span.obs]

```
constexpr static index_type rank() noexcept ;
```

Returns: Rank of the domain index space; i.e., `rank_v<ArrayType> ? rank_v<ArrayType> : 1`.

```
constexpr static index_type static_extent() noexcept ;
```

Returns: Static extent of the domain index space; i.e., `extent_v<ArrayType,0>`.

Remark: When `rank_v<ArrayType> > 1` is permitted then a new `static_extent( index_type r )` observer will be required.

```
constexpr index_type extent() const noexcept ;
```

Returns: Runtime extent of the rank-one domain index space; i.e., `extent_v<ArrayType,0> ? extent_v<ArrayType,0> : size()`.

Remark: When `rank_v<ArrayType> > 1` is permitted then a new `extent( index_type r )` observer will be required.

```
constexpr index_type size() const noexcept ;
```

Returns: Number of elements in the codomain.

```
constexpr index_type size_bytes() const noexcept ;
```

Returns: Number of bytes used for the object representation of all elements in the codomain.

Changes to span iterator support [span.iterators]

```
iterator begin() const noexcept ;
```

Returns: When `data() != nullptr` an iterator referring to the element in the codomain with the smallest address. When `data() == nullptr` then `begin() == end()`.

```
iterator end() const noexcept ;
```

Returns: When `data() != nullptr` an iterator such that `--end()` refers to the element in the codomain with the largest address. When `data() == nullptr` then `begin() == end()`.

Changes to span comparison operators [span.comparison]

```
template< class lhsArrayType , typename ... lhsProperties  
    , class rhsArrayType , typename ... rhsProperties >  
constexpr bool operator OP ( const span<lhsArrayType, lhsProperties...> &  
    , const span<rhsArrayType, rhsProperties...> & );
```

Remark: The spans *may* be comparable even when `remove_all_extents_t<lhsArrayType>` and `remove_all_extents_t<rhsArrayType>` are different types (e.g., have different cv qualification) or `lhsProperties...` and `rhsProperties...` are different.